SOC Final Report

Architecture Summary

This project has helped me understand the layers of abstraction in computer architecture. What essentially happens is that we use something called a Hardware Description Language like VHDL to provide a structural description of our cpu. This language goes through RTL (Register transfer language) synthesis which basically uses EDA tools to parse the HDL syntax and give us a representation of the code in general logic gates muxes and flip flops. The compiler uses Boolean minimization algorithms to shrink the FSM and ALU logic, and after that from my understanding there are routing tools which map elements to specific locations on wafers to ensure there aren't any delays in signals causing timing violations. Lets get into the WashU-2 design now

Instruction Set Architecture: WashU-2 follows a multi-cycle processor architecture, each instruction being encoded in 16 bits with the top 4 bits allocated for opcode, and the lower 12 bits for constants or memory addresses. It consists of 14 instructions spanning various types such as DLoad,ILoad and similar store instructions to access memory either directly or indirectly. The indirect path also enables the concept of arrays since the memory address in the instruction acts like a pointer. It also has arithmetic instructions such as negate,add and logical instructions such as And and conditional branches which change the program counter depending on various flags.

Datapath and Registers: The processor manges routing using a 16 bit tri-stated bus driver by a 7 bit one hot multiplexer. It drives one source at a time but any register can latch to the bus data.

There are five 16 bit registers:

Accumulator: It is used to store computational results.

Program Counter: It tracks the addresses of instructions.

Memory Address Register: It's a register storing address lines of the ram

Memory Buffer Register: Its like a buffer for data moving out of the RAM as well as into it.

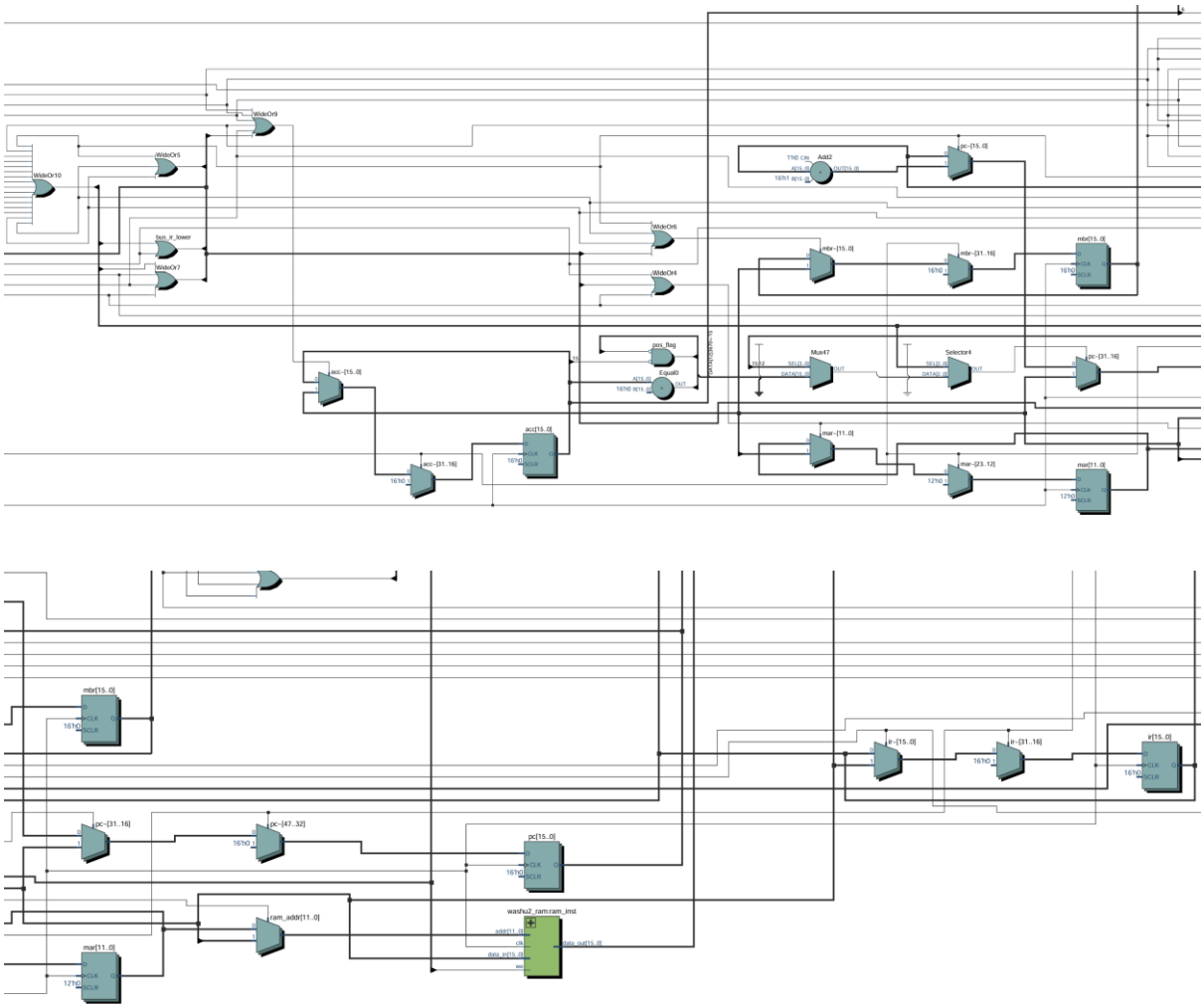
Instruction Register: It holds the instruction in it.

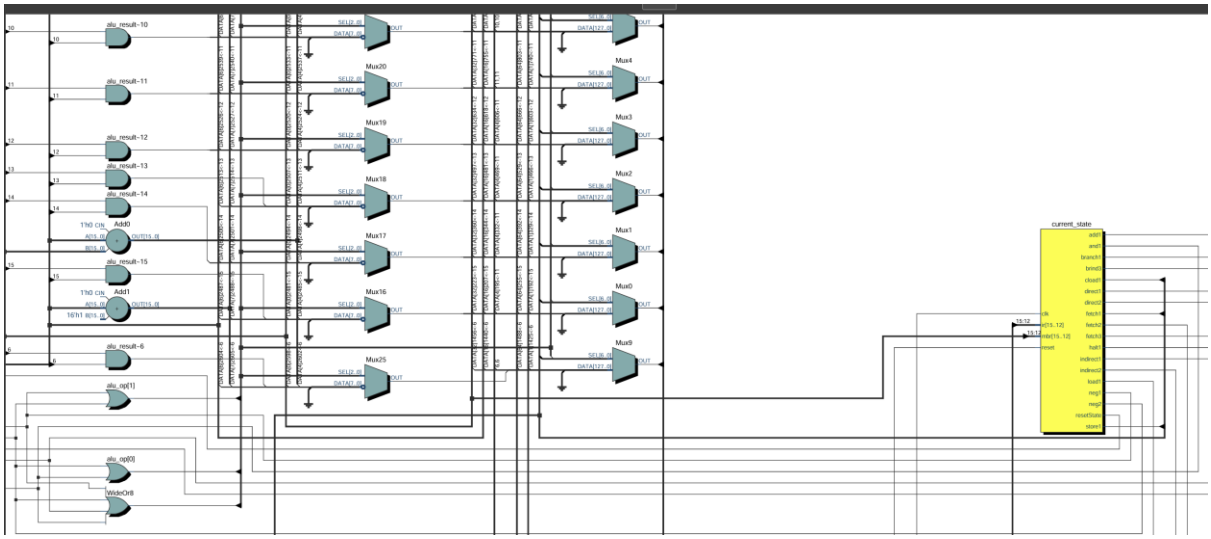
ALU: It can do basic arithmetic and logical operations such as add, and, not, increment, pass. It's a combinational process which has two operands where operand A is the accumulator and operand B is the bus.

Finite State Machine: It follows a two process framework where one clocked process handles the state change from current state to next state, while a combinational process decides the next state by looking at the opcode or some conditional flags. It also determines which control wires need to be turned on. Since we're using a bus, we can only move one piece of data at a time, that is why each move needs its own state, because of this even fetching takes 3 states.

RTL Netlist

I’ve uploaded some snippets of the netlist since a screenshot of the entire netlist isn’t legible. To see the full view, I’ve also uploaded the rtl netlist pdf.





Waveform evidence

Direct Arithmetic Branch

Program: signal ram_mem : ram_type := (

0 => x"20FF",

1 => x"8010",

2 => x"7011",

3 => x"0000",

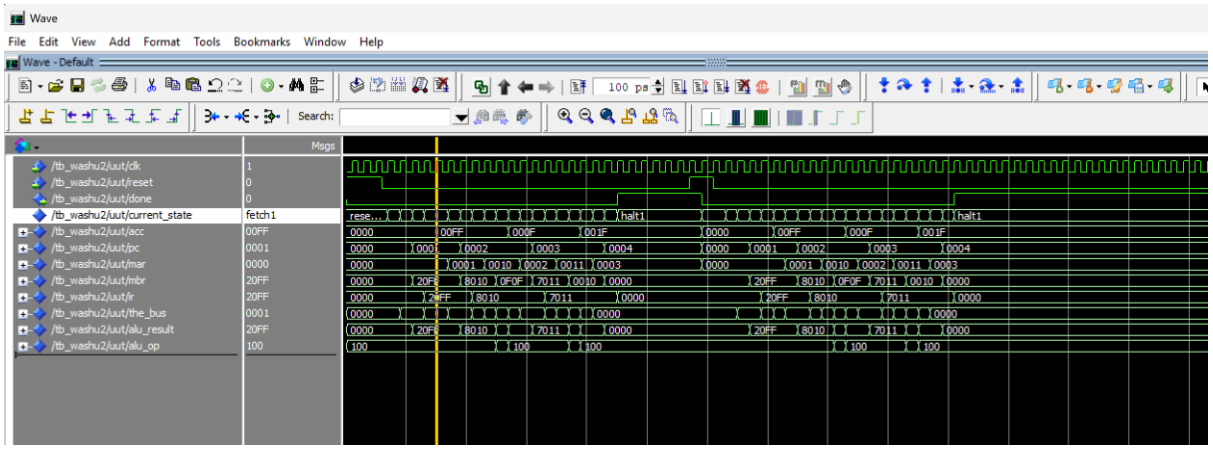
16 => x"0F0F",

17 => x"0010",

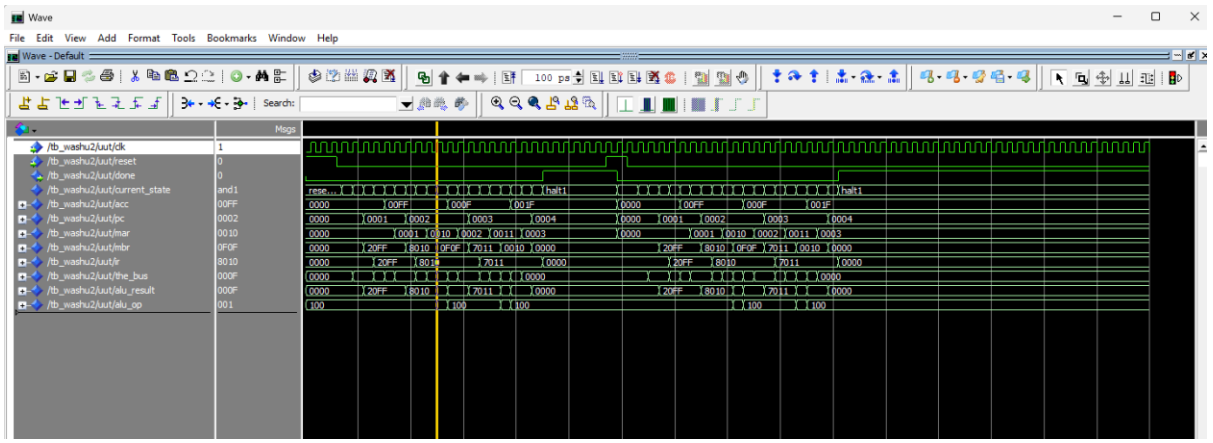
others => x"0000"

);

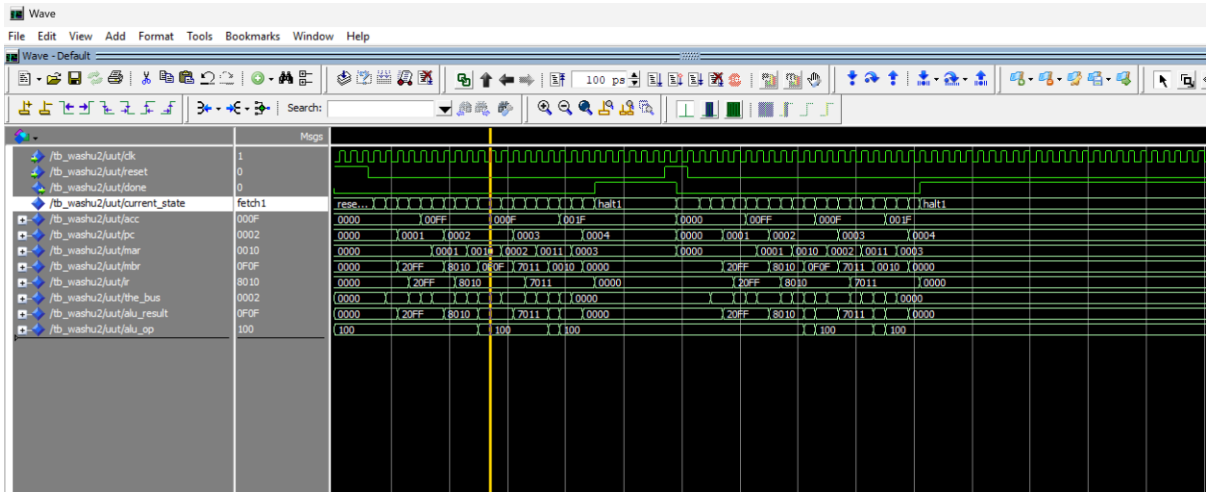
Waveforms:



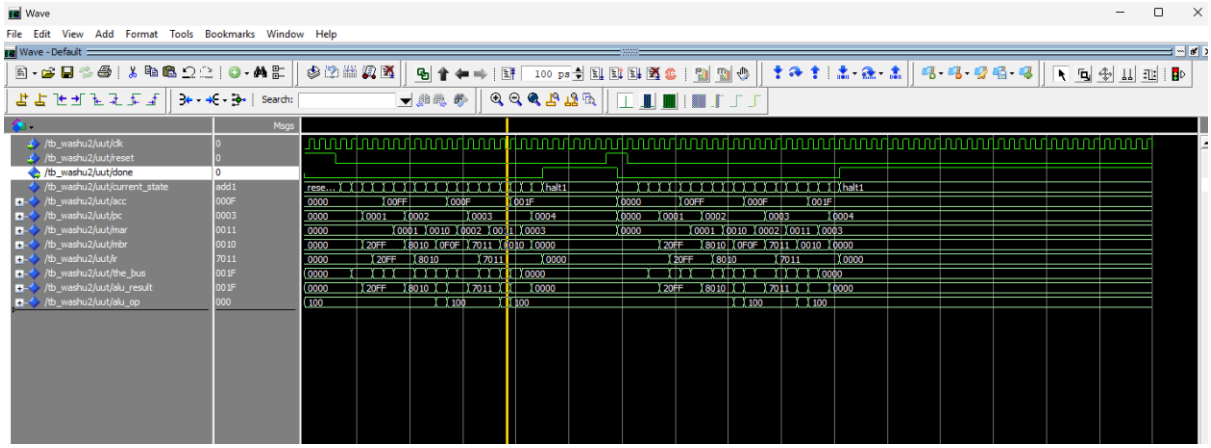
cload loaded



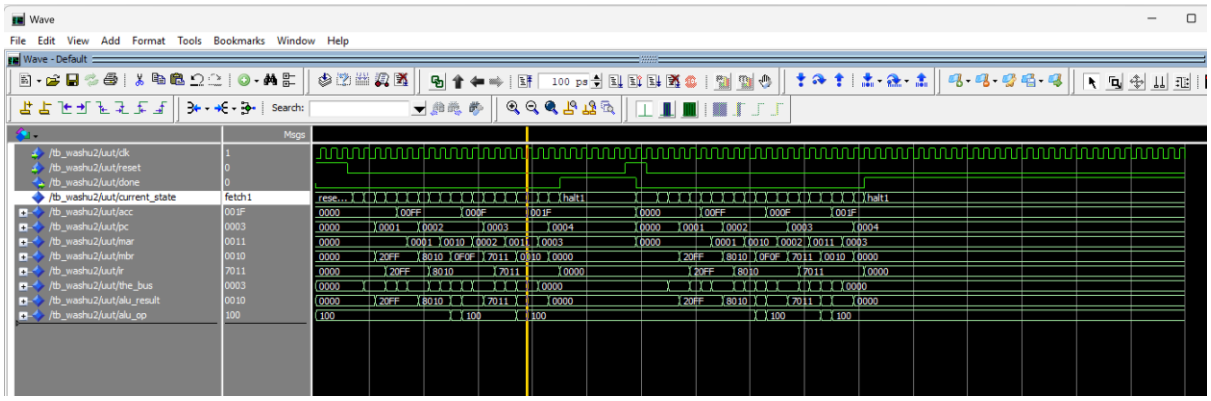
and1 ongoing



and1 executed



add1 ongoing



add1 executed

Conditional Branch

Program: signal ram_mem : ram_type := (

0 => x"2000",

1 => x"A004",

2 => x"2FFF",

3 => x"0000",

4 => x"2001",

5 => x"A002",

6 => x"B009",

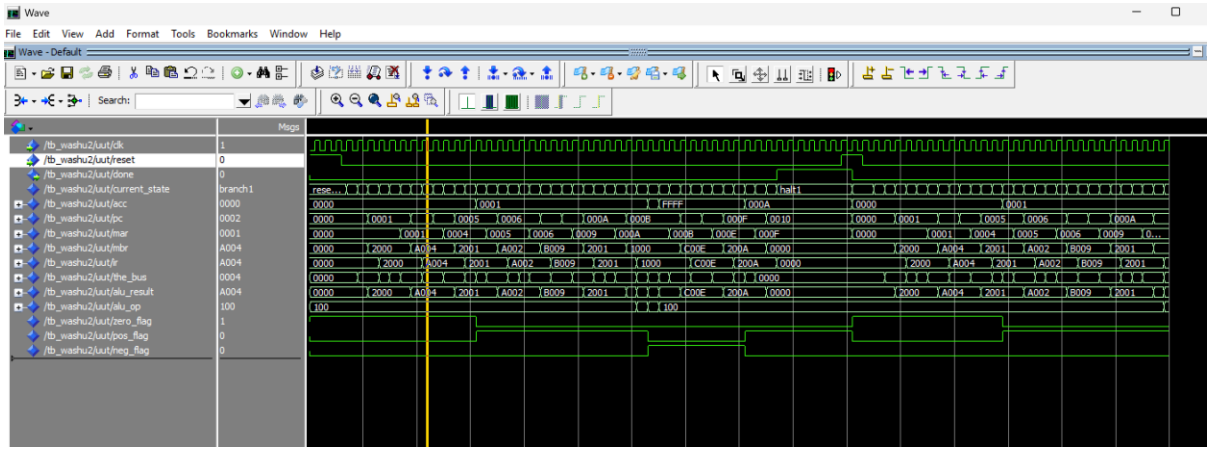
7 => x"2EEE",

8 => x"0000",

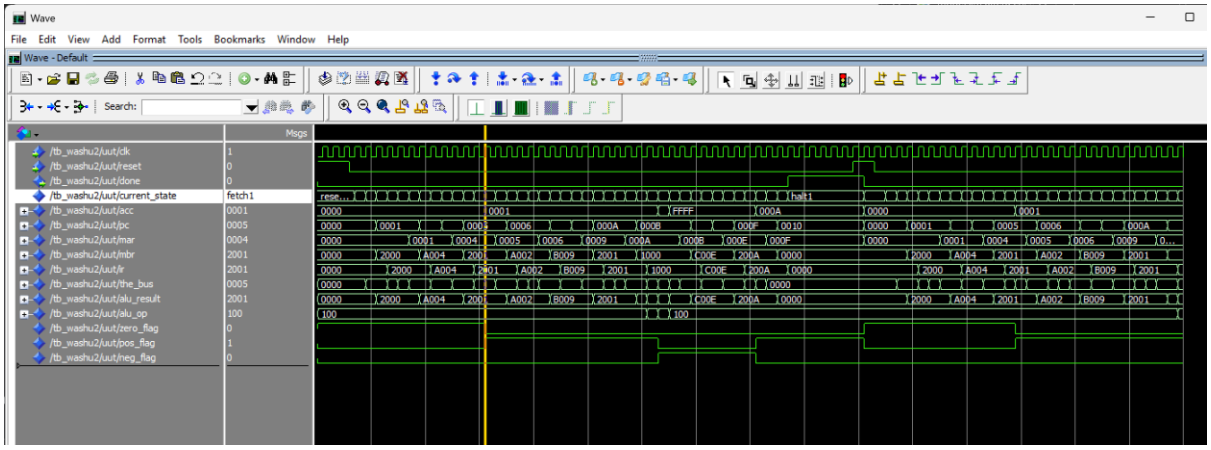
9 => x"2001",

```
        10 => x"1000",
        11 => x"C00E",
        12 => x"2DDD",
        13 => x"0000",
        14 => x"200A",
        15 => x"0000",
        others => x"0000"
    );
```

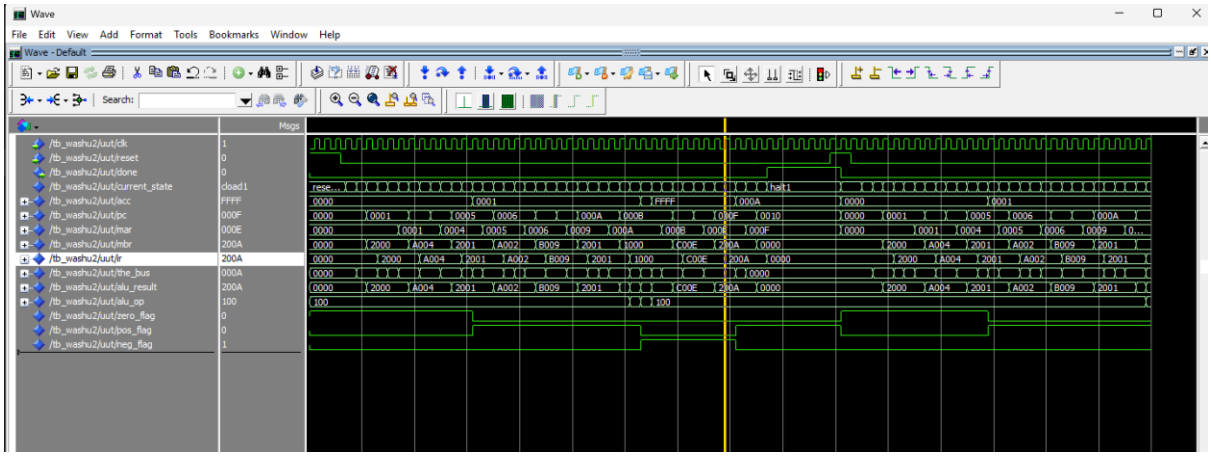
Waveforms:



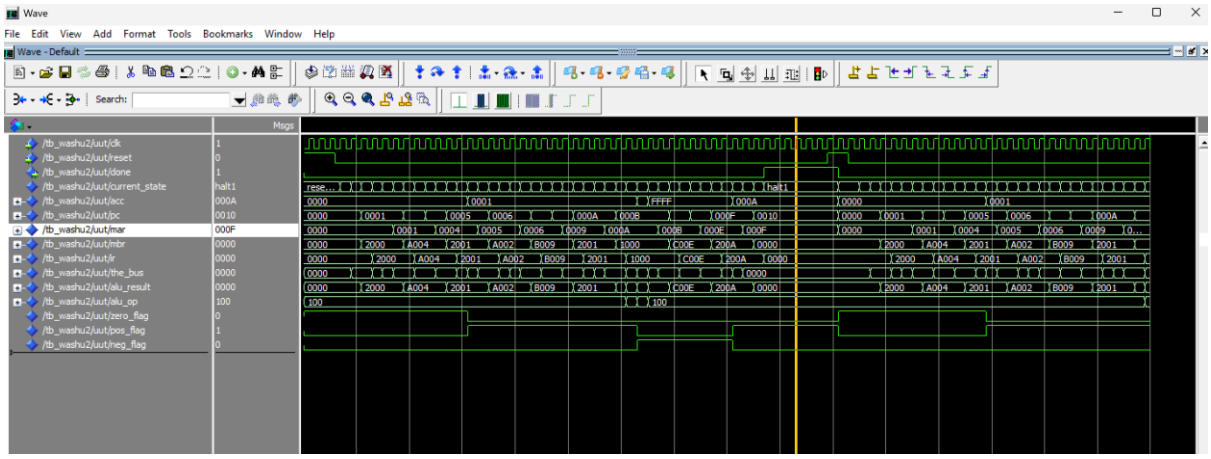
Zero Flag



Pos Flag



Neg Flag



Halt taken

Double Indirect Branch

Program: signal ram_mem : ram_type := (

0 => x"2042",

1 => x"6020",

2 => x"2000",

3 => x"5020",

4 => x"0000",

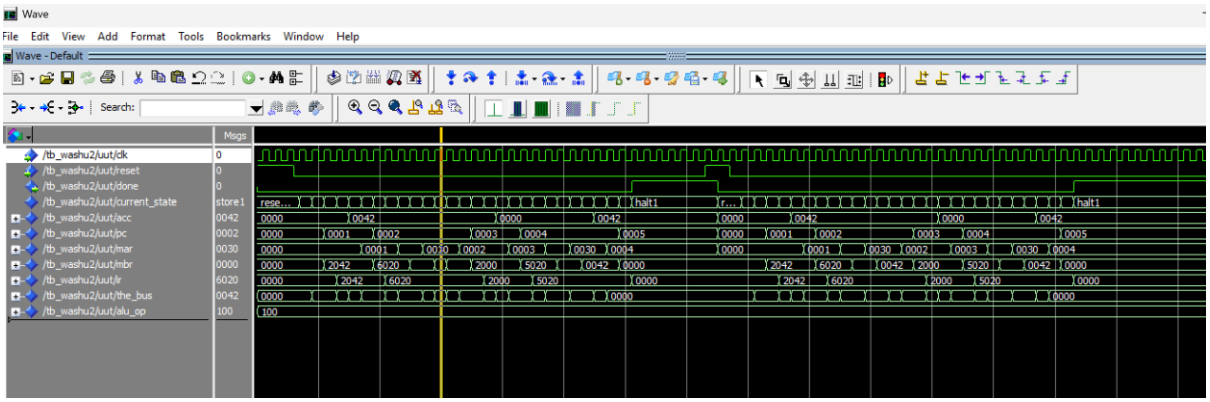
32 => x"0030",

48 => x"0000",

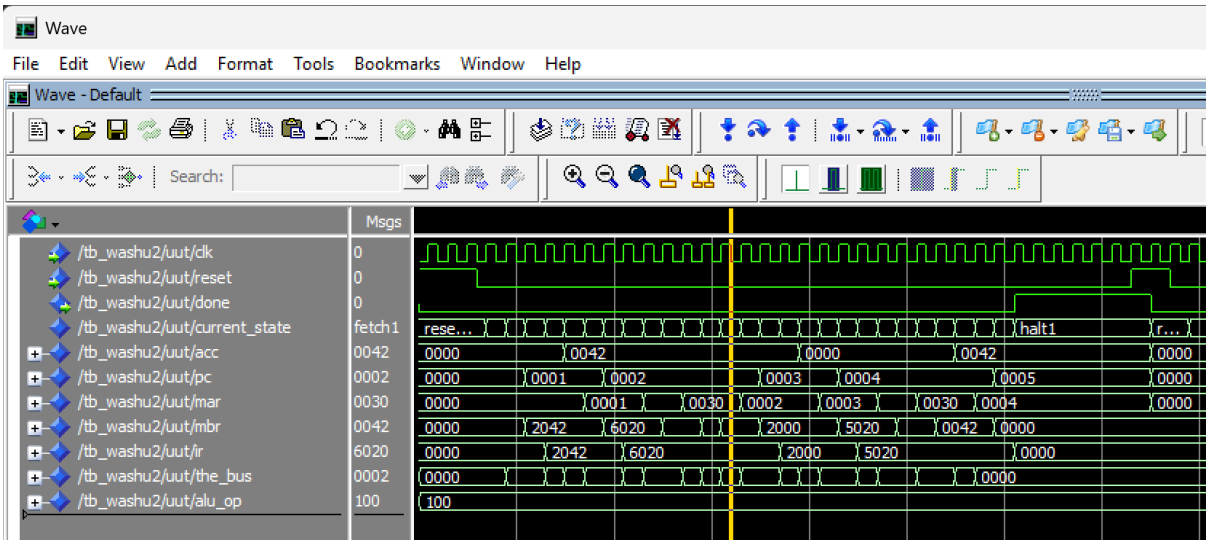
others => x"0000"

);

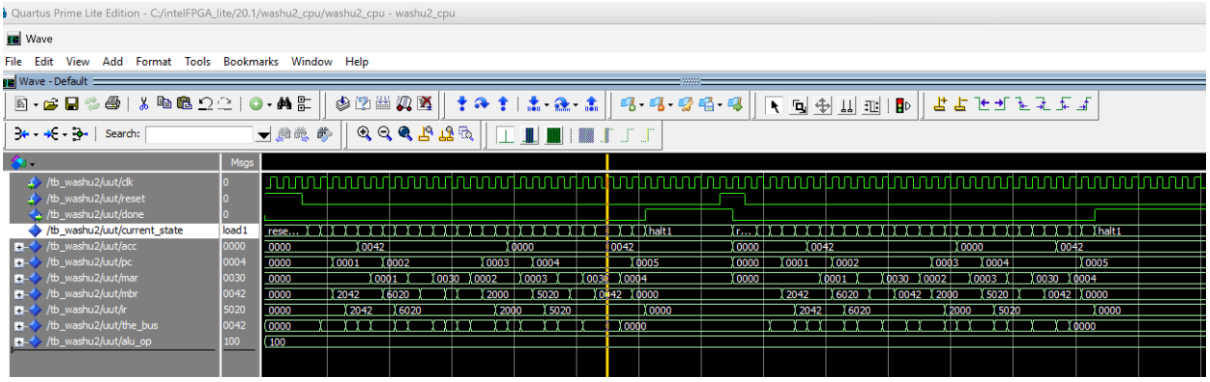
Waveforms:



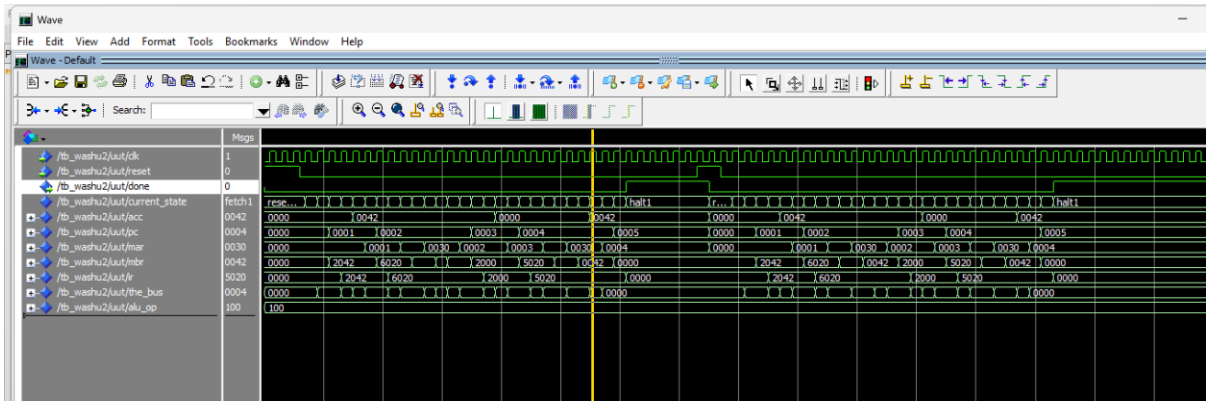
Store State



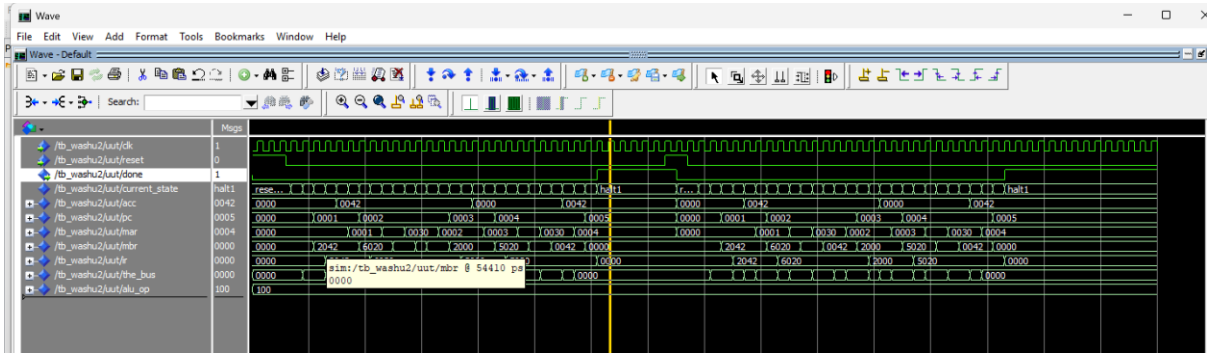
IStored



Load State



ILoaded



Halted

Integration

Program: signal ram_mem : ram_type := (

0 => x"2005",

1 => x"7040",

2 => x"A005",

3 => x"B006",

4 => x"2FFF",

5 => x"0000",

6 => x"8041",

7 => x"6042",

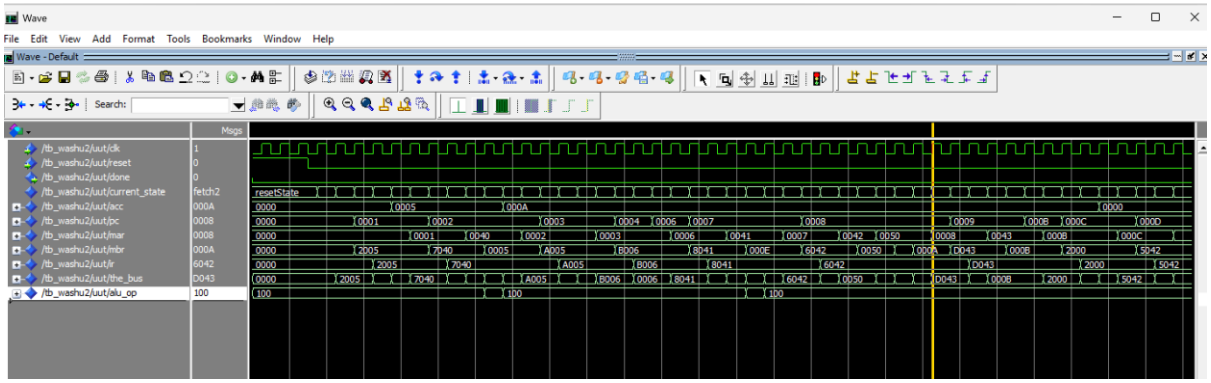
8 => x"D043",

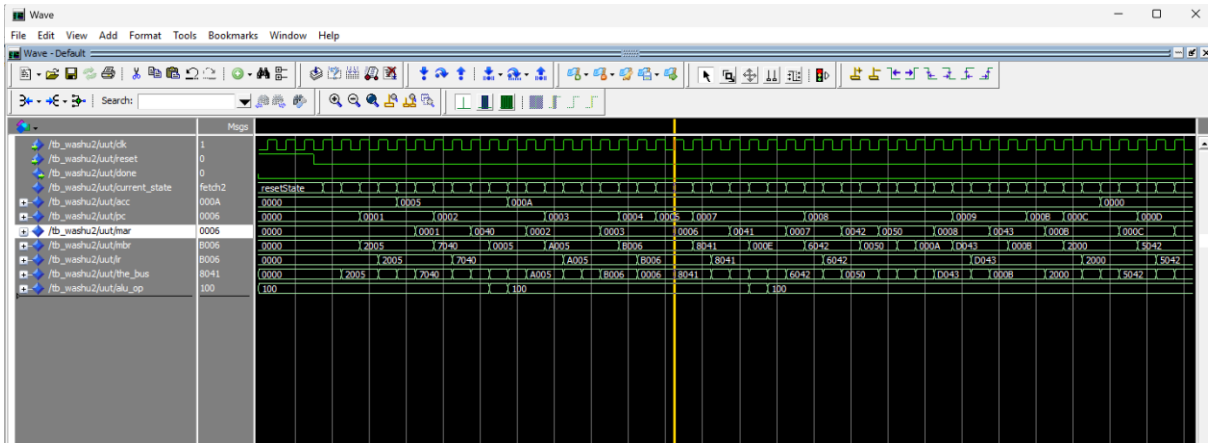
9 => x"2EEE",

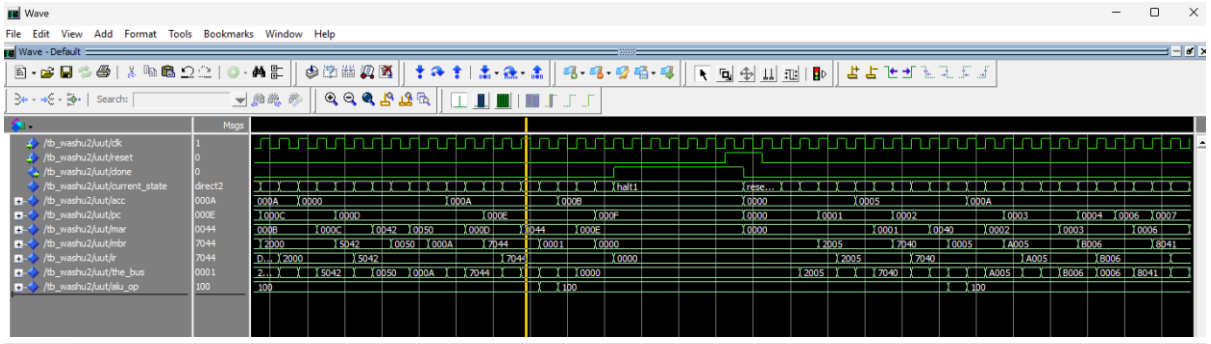
10 => x"0000",

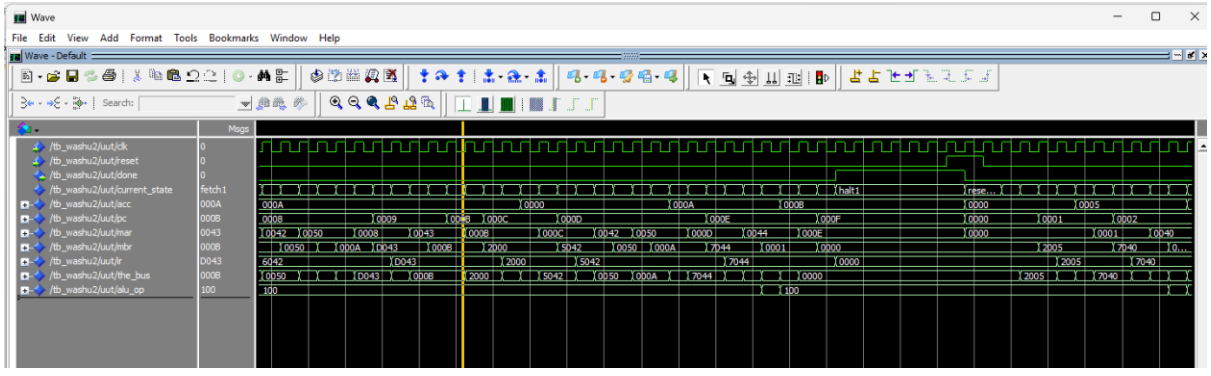
```
    11 => x"2000",
    12 => x"5042",
    13 => x"7044",
    14 => x"0000",

    64 => x"0005",
    65 => x"000E",
    66 => x"0050",
    67 => x"000B",
    68 => x"0001",

    80 => x"0000",

    others => x"0000"
);
```

Waveforms:


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Search:
Msgs
/tb_washu2/uut/clk
/tb_washu2/uut/reset
/tb_washu2/uut/done
/tb_washu2/uut/current_state
/tb_washu2/uut/acc
/tb_washu2/uut/pc
/tb_washu2/uut/mar
/tb_washu2/uut/mbr
/tb_washu2/uut/ir
/tb_washu2/uut/the_bus
/tb_washu2/uut/alu_op
resetState


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Search:
Msgs
/tb_washu2/uut/clk
/tb_washu2/uut/reset
/tb_washu2/uut/done
/tb_washu2/uut/current_state
/tb_washu2/uut/acc
/tb_washu2/uut/pc
/tb_washu2/uut/mar
/tb_washu2/uut/mbr
/tb_washu2/uut/ir
/tb_washu2/uut/the_bus
/tb_washu2/uut/alu_op
resetState


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Search:
Msgs
/tb_washu2/uut/clk
/tb_washu2/uut/reset
/tb_washu2/uut/done
/tb_washu2/uut/current_state
/tb_washu2/uut/acc
/tb_washu2/uut/pc
/tb_washu2/uut/mar
/tb_washu2/uut/mbr
/tb_washu2/uut/ir
/tb_washu2/uut/the_bus
/tb_washu2/uut/alu_op


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Search:
Msgs
/tb_washu2/uut/clk
/tb_washu2/uut/reset
/tb_washu2/uut/done
/tb_washu2/uut/current_state
/tb_washu2/uut/acc
/tb_washu2/uut/pc
/tb_washu2/uut/mar
/tb_washu2/uut/mbr
/tb_washu2/uut/ir
/tb_washu2/uut/the_bus
/tb_washu2/uut/alu_op

Bugs Encountered

- In the store state, I did not load mbr to the bus, which ended up giving the incorrect/old value to write on the ram, so I ended up porting the bus to the data_in of the ram so that the correct value is loaded in.
- I selected a MAX 10 device which struggled at inferring ram because it handled memory block initialisation differently than other fpga's like cyclone IV Basically it tried to unroll all 4096 16 bit registers into logic array blocks. I solved it by switching to another device
- In ir_addr_se I didn't properly do the sign extension which lead to a 12 bit value being assigned to a 16 bit value
  The final line was : ir_addr_se <= (15 downto 12 => ir(11)) & ir(11 downto 0);
- I also kept losing track of states and accidentally increased states at times instead of using my reduced fsm diagram
- At a certain point I was also encoding instruction incorrectly because I kept confusing the radix of all the values and was intermixing hexadecimal with decimal etc., which led to me encoding the instructions incorrectly.

Cycle – Count Table

1. HALT – 4
2. NEGATE – 5
3. CLOAD – 4
4. DLOAD – 6
5. DSTORE – 6
6. ILOAD – 8
7. ISTORE – 8
8. ADD – 6
9. AND – 6
10. BRANCH – 4
11. BRZERO – 4
12. BRPOS – 4
13. BRNEG – 4
14. BRIND - 6

Self – assessment

The hardest instruction by far was the ISTORE, because even though I had discarded the ISTORE state in my original plan for the cpu, I used it again which led to a compilation error because I didn't declare that state earlier. I also did not load the mbr register to the data bus which ended up writing the previous value stored on mbr which was incorrect, to solve this I ported the bus to the ram so that it could be updated with the correct value.